TD2 compte rendu

Nour hbaieb

Exercice2 :

Quelle est l'organisation des livraisons des containers pour minimiser le coût total de transport ?

1 . Modélisation mathématique

- **Données du problème** :

	M1	M2	M3	Stock du dépôt
D1	5	3	4	8
D2	6	7	2	9
Demande	4	5	8	17

Le stock total : 8 + 9 = 17 conteneurs et la demande totale : 4 + 5 + 8 = 17 conteneurs d'où le problème est équilibré

- **Les variables de décision** : On note (Xij) le nombre de conteneurs envoyés du dépôt (Di) au magasin (Mj) :

  X11 : nombre de conteneurs envoyés de D1 vers M1

  X12 : nombre de conteneurs envoyés de D1 vers M2

  X13 : nombre de conteneurs envoyés de D1 vers M3

  X21 : nombre de conteneurs envoyés de D2 vers M1

  X22 : nombre de conteneurs envoyés de D2 vers M2

  X23 : nombre de conteneurs envoyés de D2 vers M3

- **La fonction objectif** : Le coût total à minimiser

  Min Z =5**X11* + *3*X12* + *4*X13* + *6*X21* + *7*X22* + *2**X23

- **Les contraintes** :
  - **Contraintes des dépôts** : tout le stock de chaque dépôt doit être expédié (car on a un stock total : 8 + 9 = 17 conteneurs et la demande totale : 4 + 5 + 8 = 17 conteneurs) donc on obtient :
    X11 + X12 + X13 =8
    X21 + X22 + X23 =9
  - **Contraintes des magasins** : chaque magasin doit recevoir sa demande exacte :
    X11 + X21 = 4
    X12 + X22 = 5
    X13 + X23 = 8
  - **Contraintes des variables** : doivent être des entiers positifs ou nuls : (Xij) ∈ ℕ

2 .Proposition d'une solution basée sur la méthode de recherche taboue

J'ai proposé une solution pour résoudre le problème de transport en utilisant la méthode de **recherche taboue** , dont l'objectif est de minimiser le coût total du transport tout en respectant les contraintes d'offre et de demande.

2.1. Initialisation : J'ai commencé par une solution initiale réalisable de coût total égal à 61. J'ai fixé le nombre maximal d'itérations à 10 et la taille de la liste taboue à 3 mouvements.

2.2. Génération du voisinage : À chaque itération, j'ai généré des solutions voisines en modifiant les quantités transportées. Par exemple, le mouvement (M3-1, M2+1) consiste à diminuer d'une unité la quantité envoyée de D1 vers M3 et à augmenter d'une unité celle envoyée vers M2. Les quantités de D2 sont ajustées afin de satisfaire les demandes des magasins.

2.3. Calcul du coût : Après chaque mouvement, j'ai calculé le nouveau coût total et je l'ai comparé au coût de la solution précédente afin d'évaluer l'amélioration obtenue.

2.4. Gestion de la liste taboue : Après l'exécution d'un mouvement, j'ai ajouté son mouvement inverse à la liste taboue pour empêcher un retour immédiat à la solution précédente. La liste peut contenir au maximum trois mouvements ; lorsque sa capacité est atteinte, le mouvement le plus ancien est retiré.

2.5. Répétition des itérations : J'ai répété ce processus pendant 10 itérations, en sélectionnant les mouvements autorisés et en conservant la meilleure solution trouvée, même lorsque le coût de la solution courante augmente

temporairement.

2.6. Résultat obtenu : À partir de la solution initiale de coût 61, les premières améliorations permettent d'obtenir un coût de 55, puis de 52. La meilleure solution obtenue est :

	M1	M2	M3
D1	3	5	0
D2	1	0	8

Le coût total correspondant est :

Min z =5*3 + 3*5 + 4*0 + 6*1 + 7*0+ 2*8 = 52

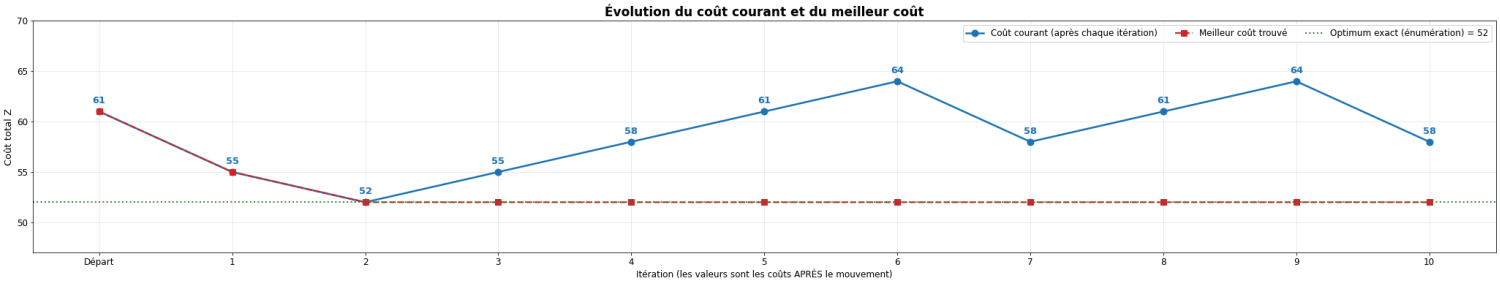
Conclusion : La méthode de recherche taboue m'a permis d'explorer différentes solutions réalisables et de réduire le coût total du transport de 61 à 52, soit une diminution d'environ 14,75 %. La solution finale respecte toutes les contraintes d'offre et de demande et correspond à l'optimum de ce problème.

3 . Analyse de la figure : Application de la recherche taboue

La figure ci-dessous illustre l'application de la méthode de recherche taboue décrite précédemment. Elle présente les différentes étapes de l'algorithme, depuis la solution initiale de coût 61 jusqu'aux solutions obtenues au cours des 10 itérations. À chaque itération, les mouvements possibles sont évalués et la liste taboue, de taille 3, permet d'éviter les retours immédiats aux solutions précédentes en mémorisant les mouvements inverses. Comme le montre la figure, les premières itérations permettent de réduire le coût à 55, puis à 52. La courbe représente l'évolution du coût courant et du meilleur coût obtenu. Ainsi, cette figure illustre comment la recherche taboue explore le voisinage des solutions et conserve la meilleure solution trouvée, de coût total égal à 52, tout en respectant les contraintes d'offre et de demande.
voici le repository qui contient le code python de la proposition :
TD-H-MH/exercice2/main.py at master · nourhbaieb88/TD-H-MH

Solution initiale D1 = (4, 3, 1), Z = 61 | 10 itérations | liste taboue de 3 cases

Évolution du coût courant et du meilleur coût



Comment lire un mouvement ? `M3 −1` `M2 +1` = D1 déplace 1 container de M3 vers M2 (il livre 1 de moins à M3 et 1 de plus à M2).

D2 compense automatiquement : les demandes des magasins restent toujours satisfaites. Chaque mouvement change le coût total Z.

Liste taboue : après un mouvement, on INTERDIT le mouvement contraire (remettre le container là où il était) pendant 3 itérations, pour ne pas tourner en rond.

Solution courante (au DÉBUT de l'itération)	Mouvements possibles (voisins), classés par coût croissant (on choisit le moins cher des mouvements NON interdits (même s'il est plus cher que la solution courante))	Mouvement choisi (et mise à jour de la liste taboue)	LISTE TABOUE (3 cases) (mouvements interdits pendant l'itération (case 1 = la plus récente))
Itération 1 D1 = (4, 3, 1) D2 = (0, 2, 7) **Z = 61 (meilleur : 61)**	n°1 `M3 −1` `M2 +1` D1 = (4, 4, 0) **Z = 55** ✓ CHOISI n°2 `M1 −1` `M2 +1` D1 = (3, 4, 1) **Z = 58** autorisé n°3 `M1 −1` `M3 +1` D1 = (3, 3, 2) **Z = 64** autorisé n°4 `M2 −1` `M3 +1` D1 = (4, 2, 2) **Z = 67** autorisé	**Choisi :** `M3 −1` `M2 +1` D1 déplace 1 container de M3 vers M2 **Z : 61 → 55 (-6) D1 = (4, 4, 0)** **Interdit ensuite : 1 container de M2 vers M3** aucune case ne sort de la liste	Case 1 (récente) libre Case 2 libre Case 3 (ancienne) libre
Itération 2 D1 = (4, 4, 0) D2 = (0, 1, 8) **Z = 55 (meilleur : 55)**	n°1 `M1 −1` `M2 +1` D1 = (3, 5, 0) **Z = 52** ✓ CHOISI n°2 `M1 −1` `M3 +1` D1 = (3, 4, 1) **Z = 58** autorisé n°3 `M2 −1` `M3 +1` D1 = (4, 3, 1) **Z = 61** ✗ INTERDIT (case 1)	**Choisi :** `M1 −1` `M2 +1` D1 déplace 1 container de M1 vers M2 **Z : 55 → 52 (-3) D1 = (3, 5, 0)** **Interdit ensuite : 1 container de M2 vers M1** aucune case ne sort de la liste	Case 1 (récente) INTERDIT `M2 −1` `M3 +1` **1 container de M2 vers M3** annule le mouvement de l'it. 1 Case 2 libre Case 3 (ancienne) libre
Itération 3 D1 = (3, 5, 0) D2 = (1, 0, 8) **Z = 52 (meilleur : 52)**	n°1 `M1 −1` `M3 +1` D1 = (2, 5, 1) **Z = 55** ✓ CHOISI n°2 `M2 −1` `M1 +1` D1 = (4, 4, 0) **Z = 55** ✗ INTERDIT (case 1) n°3 `M2 −1` `M3 +1` D1 = (3, 4, 1) **Z = 58** ✗ INTERDIT (case 2)	**Choisi :** `M1 −1` `M3 +1` D1 déplace 1 container de M1 vers M3 **Z : 52 → 55 (+3) D1 = (2, 5, 1)** **Interdit ensuite : 1 container de M3 vers M1** aucune case ne sort de la liste	Case 1 (récente) INTERDIT `M2 −1` `M1 +1` **1 container de M2 vers M1** annule le mouvement de l'it. 2 Case 2 INTERDIT `M2 −1` `M3 +1` **1 container de M2 vers M3** annule le mouvement de l'it. 1 Case 3 (ancienne) libre
Itération 4 D1 = (2, 5, 1) D2 = (2, 0, 7) **Z = 55 (meilleur : 52)**	n°1 `M3 −1` `M1 +1` D1 = (3, 5, 0) **Z = 52** ✗ INTERDIT (case 1) n°2 `M1 −1` `M3 +1` D1 = (1, 5, 2) **Z = 58** ✓ CHOISI n°3 `M2 −1` `M1 +1` D1 = (3, 4, 1) **Z = 58** ✗ INTERDIT (case 2) n°4 `M2 −1` `M3 +1` D1 = (2, 4, 2) **Z = 61** ✗ INTERDIT (case 3)	**Choisi :** `M1 −1` `M3 +1` D1 déplace 1 container de M1 vers M3 **Z : 55 → 58 (+3) D1 = (1, 5, 2)** **Interdit ensuite : 1 container de M3 vers M1** sort de la liste : 1 container de M2 vers M3	Case 1 (récente) INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 3 Case 2 INTERDIT `M2 −1` `M1 +1` **1 container de M2 vers M1** annule le mouvement de l'it. 2 Case 3 (ancienne) INTERDIT `M2 −1` `M3 +1` **1 container de M2 vers M3** annule le mouvement de l'it. 1
Itération 5 D1 = (1, 5, 2) D2 = (3, 0, 6) **Z = 58 (meilleur : 52)**	n°1 `M3 −1` `M1 +1` D1 = (2, 5, 1) **Z = 55** ✗ INTERDIT (case 1) n°2 `M1 −1` `M3 +1` D1 = (0, 5, 3) **Z = 61** ✓ CHOISI n°3 `M2 −1` `M1 +1` D1 = (2, 4, 2) **Z = 61** ✗ INTERDIT (case 3) n°4 `M2 −1` `M3 +1` D1 = (1, 4, 3) **Z = 64** autorisé	**Choisi :** `M1 −1` `M3 +1` D1 déplace 1 container de M1 vers M3 **Z : 58 → 61 (+3) D1 = (0, 5, 3)** **Interdit ensuite : 1 container de M3 vers M1** sort de la liste : 1 container de M2 vers M1	Case 1 (récente) INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 4 Case 2 INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 3 Case 3 (ancienne) INTERDIT `M2 −1` `M1 +1` **1 container de M2 vers M1** annule le mouvement de l'it. 2
Itération 6 D1 = (0, 5, 3) D2 = (4, 0, 5) **Z = 61 (meilleur : 52)**	n°1 `M3 −1` `M1 +1` D1 = (1, 5, 2) **Z = 58** ✗ INTERDIT (case 1) n°2 `M2 −1` `M1 +1` D1 = (1, 4, 3) **Z = 64** ✓ CHOISI n°3 `M2 −1` `M3 +1` D1 = (0, 4, 4) **Z = 67** autorisé	**Choisi :** `M2 −1` `M1 +1` D1 déplace 1 container de M2 vers M1 **Z : 61 → 64 (+3) D1 = (1, 4, 3)** **Interdit ensuite : 1 container de M1 vers M2** sort de la liste : 1 container de M3 vers M1	Case 1 (récente) INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 5 Case 2 INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 4 Case 3 (ancienne) INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 3
Itération 7 D1 = (1, 4, 3) D2 = (3, 1, 5) **Z = 64 (meilleur : 52)**	n°1 `M3 −1` `M2 +1` D1 = (1, 5, 2) **Z = 58** ✓ CHOISI n°2 `M1 −1` `M2 +1` D1 = (0, 5, 3) **Z = 61** ✗ INTERDIT (case 1) n°3 `M3 −1` `M1 +1` D1 = (2, 4, 2) **Z = 61** ✗ INTERDIT (case 2) n°4 `M1 −1` `M3 +1` D1 = (0, 4, 4) **Z = 67** autorisé n°5 `M2 −1` `M1 +1` D1 = (2, 3, 3) **Z = 67** autorisé n°6 `M2 −1` `M3 +1` D1 = (1, 3, 4) **Z = 70** autorisé	**Choisi :** `M3 −1` `M2 +1` D1 déplace 1 container de M3 vers M2 **Z : 64 → 58 (-6) D1 = (1, 5, 2)** **Interdit ensuite : 1 container de M2 vers M3** sort de la liste : 1 container de M3 vers M1	Case 1 (récente) INTERDIT `M1 −1` `M2 +1` **1 container de M1 vers M2** annule le mouvement de l'it. 6 Case 2 INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 5 Case 3 (ancienne) INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 4
Itération 8 D1 = (1, 5, 2) D2 = (3, 0, 6) **Z = 58 (meilleur : 52)**	n°1 `M3 −1` `M1 +1` D1 = (2, 5, 1) **Z = 55** ✗ INTERDIT (case 3) n°2 `M1 −1` `M3 +1` D1 = (0, 5, 3) **Z = 61** ✓ CHOISI n°3 `M2 −1` `M1 +1` D1 = (2, 4, 2) **Z = 61** autorisé n°4 `M2 −1` `M3 +1` D1 = (1, 4, 3) **Z = 64** ✗ INTERDIT (case 1)	**Choisi :** `M1 −1` `M3 +1` D1 déplace 1 container de M1 vers M3 **Z : 58 → 61 (+3) D1 = (0, 5, 3)** **Interdit ensuite : 1 container de M3 vers M1** sort de la liste : 1 container de M3 vers M1	Case 1 (récente) INTERDIT `M2 −1` `M3 +1` **1 container de M2 vers M3** annule le mouvement de l'it. 7 Case 2 INTERDIT `M1 −1` `M2 +1` **1 container de M1 vers M2** annule le mouvement de l'it. 6 Case 3 (ancienne) INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 5
Itération 9 D1 = (0, 5, 3) D2 = (4, 0, 5) **Z = 61 (meilleur : 52)**	n°1 `M3 −1` `M1 +1` D1 = (1, 5, 2) **Z = 58** ✗ INTERDIT (case 1) n°2 `M2 −1` `M1 +1` D1 = (1, 4, 3) **Z = 64** ✓ CHOISI n°3 `M2 −1` `M3 +1` D1 = (0, 4, 4) **Z = 67** ✗ INTERDIT (case 2)	**Choisi :** `M2 −1` `M1 +1` D1 déplace 1 container de M2 vers M1 **Z : 61 → 64 (+3) D1 = (1, 4, 3)** **Interdit ensuite : 1 container de M1 vers M2** sort de la liste : 1 container de M1 vers M2	Case 1 (récente) INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 8 Case 2 INTERDIT `M2 −1` `M3 +1` **1 container de M2 vers M3** annule le mouvement de l'it. 7 Case 3 (ancienne) INTERDIT `M1 −1` `M2 +1` **1 container de M1 vers M2** annule le mouvement de l'it. 6
Itération 10 D1 = (1, 4, 3) D2 = (3, 1, 5) **Z = 64 (meilleur : 52)**	n°1 `M3 −1` `M2 +1` D1 = (1, 5, 2) **Z = 58** ✓ CHOISI n°2 `M1 −1` `M2 +1` D1 = (0, 5, 3) **Z = 61** ✗ INTERDIT (case 1) n°3 `M3 −1` `M1 +1` D1 = (2, 4, 2) **Z = 61** ✗ INTERDIT (case 2) n°4 `M1 −1` `M3 +1` D1 = (0, 4, 4) **Z = 67** autorisé n°5 `M2 −1` `M1 +1` D1 = (2, 3, 3) **Z = 67** autorisé n°6 `M2 −1` `M3 +1` D1 = (1, 3, 4) **Z = 70** ✗ INTERDIT (case 3)	**Choisi :** `M3 −1` `M2 +1` D1 déplace 1 container de M3 vers M2 **Z : 64 → 58 (-6) D1 = (1, 5, 2)** **Interdit ensuite : 1 container de M2 vers M3** sort de la liste : 1 container de M2 vers M3	Case 1 (récente) INTERDIT `M1 −1` `M2 +1` **1 container de M1 vers M2** annule le mouvement de l'it. 9 Case 2 INTERDIT `M3 −1` `M1 +1` **1 container de M3 vers M1** annule le mouvement de l'it. 8 Case 3 (ancienne) INTERDIT `M2 −1` `M3 +1` **1 container de M2 vers M3** annule le mouvement de l'it. 7

Résultat : meilleure solution D1 = (3, 5, 0), D2 = (1, 0, 8), Z = 52 (trouvée à l'itération 2)

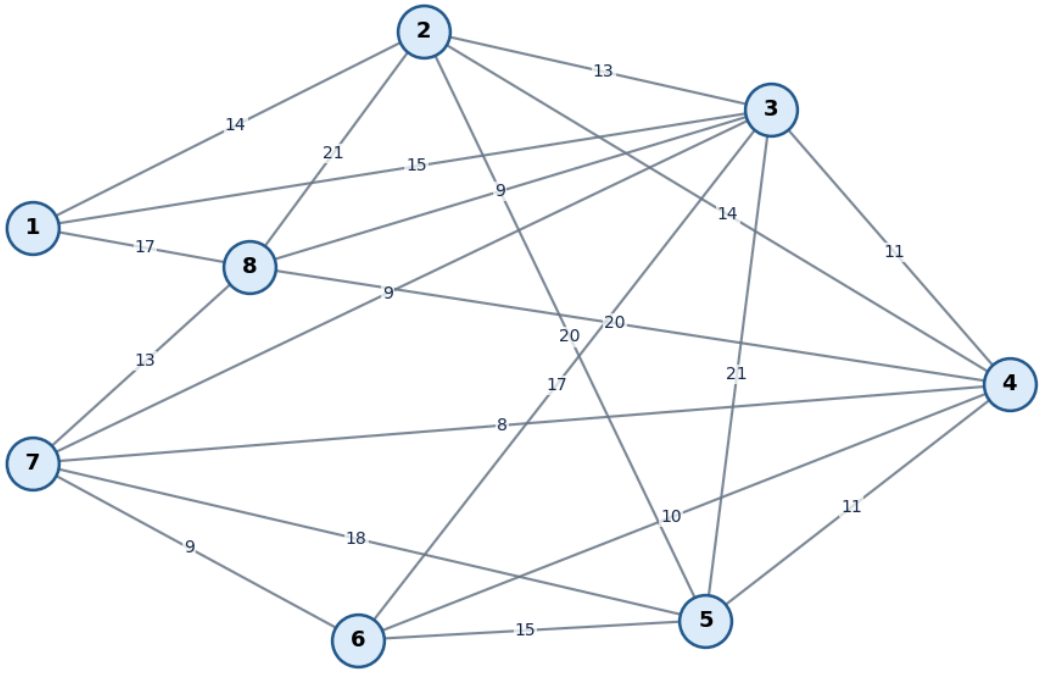
Énumération complète : 29 solutions réalisables, optimum Z = 52 → identique à l'optimum exact. Solution courante après l'itération 10 : D1 = (1, 5, 2), Z = 58.

Mouvement choisi (le moins cher des mouvements autorisés) — Mouvement autorisé mais moins bon — Case occupée de la liste taboue
Mouvement interdit (il annule un mouvement récent) — Aspiration : interdit mais meilleur que le meilleur Z (jamais utilisée ici) — Mouvement choisi + mise à jour de la liste

Pastille rouge « Mk −1 » = D1 livre 1 container de moins à Mk ; pastille verte « Mj +1 » = D1 en livre 1 de plus à Mj. Seuls les mouvements réalisables sont générés. Si la liste a déjà 3 cases, la plus ancienne sort.

Exercice 3 :

1 . Représentation graphique du problème



2 . Résolution du problème du voyageur de commerce par le recherche taboue :

2.1 Initialisation :

- **Solution initiale** : (1-2-3-4-5-6-7-8-1)
- **Le voisinage :** échanger la position de deux villes (la ville 1 reste fixe). Cela donne 21 voisins possibles
- **Taille de la liste taboue** : 3 mouvements
- **Nombre d'itérations** : 10

2.2 Déroulement des itérations :

À chaque itération, on effectue un échange entre deux villes pour construire un nouveau circuit. On calcule ensuite son coût, on le compare au meilleur coût obtenu depuis le début et on enregistre l'échange dans la liste taboue.

La liste taboue possède trois cases. Elle mémorise les trois derniers échanges effectués afin d'éviter de répéter immédiatement les mêmes mouvements.

- **Itération 1 :** on échange les villes 2 et 3. Le nouveau circuit a un coût de 107. Comme ce coût est supérieur à 103, le meilleur coût reste 103.

L'échange 2↔3 est ajouté à la liste taboue.

- **Itération 2 :** on échange les villes 4 et 5. Le coût du circuit devient 108. Le meilleur coût reste 103. La liste taboue contient maintenant les échanges 4↔5 et 2↔3.
- **Itération 3 :** on échange les villes 4 et 6. Le coût atteint 111. Le meilleur coût reste 103. La liste taboue contient les trois échanges récents : 4↔6, 4↔5 et 2↔3.
- **Itération 4 :** on effectue l'échange 3↔8. Le coût obtenu est 115. L'échange le plus ancien est retiré de la liste, qui conserve les trois mouvements les plus récents.
- **Itération 5 :** on échange les villes 2 et 7. Le coût reste égal à 115. La liste taboue est mise à jour et contient les échanges 2↔7, 3↔8 et 4↔6.

Tableau des 5 itérations :

Itération	Circuit courant	Coût	Échange choisi	Meilleur coût	Liste taboue après l'itération (case 1 · 2 · 3)
0	1-2-3-4-5-6-7-8-1	103	-	103	vide · vide · vide
1	1-3-2-4-5-6-7-8-1	107	2↔3	103	2↔3 · vide · vide
2	1-3-2-5-4-6-7-8-1	108	4↔5	103	4↔5 · 2↔3 · vide
3	1-3-2-5-6-4-7-8-1	111	4↔6	103	4↔6 · 4↔5 · 2↔3
4	1-8-2-5-6-4-7-3-1	115	3↔8	103	3↔8 · 4↔6 · 4↔5
5	1-8-7-5-6-4-2-3-1	115	2↔7	103	2↔7 · 3↔8 · 4↔6

1.3 Résultat obtenu :

Après cinq itérations, le meilleur coût reste égal à 103, car aucun circuit obtenu n'améliore la solution initiale. La recherche taboue permet d'explorer de nouveaux circuits tout en évitant de répéter les mouvements récents. Ce résultat n'est pas nécessairement optimal

Le code python est sur le GitHub :
TD-H-MH/exercice3/tboue.py at master · nourhbaieb88/TD-H-MH

3 . Résolution du problème du voyageur de commerce par le recuit simulé

3.1 Hyperparamètres choisis

Hyperparamètre	Symbole	Valeur
Température initiale	T_0	10
Diminution de température	α	2
Nombre d'itérations	N	5
Nombre aléatoire fixe	r	0,5
Température minimale	Tmin	2

3.2 Formule de refroidissement

$$T_{K+1} = \alpha \times T_k$$

3.3 Évolution du chemin et de la température au cours des cinq itérations du recuit simulé

Niveau	Température T	Chemin candidat	Coût C
Initial	—	1 → 2 → 5 → 4 → 6 → 7 → 3 → 8 → 1	99
1	10	1 → 2 → 4 → 5 → 6 → 7 → 3 → 8 → 1	98
2	8	1 → 2 → 5 → 4 → 6 → 7 → 3 → 8 → 1	99
3	6	1 → 2 → 5 → 6 → 4 → 7 → 3 → 8 → 1	102
4	4	1 → 2 → 4 → 5 → 6 → 7 → 3 → 8 → 1	98
5	2	1 → 2 → 3 → 5 → 6 → 7 → 4 → 8 → 1	117

3.4 Étapes de décision et d'acceptation

Pour chaque itération, la décision d'acceptation du chemin candidat est prise selon les étapes suivantes :

- **Calcul de la variation du coût :** calculer la différence entre le coût du chemin candidat et celui du chemin courant :
  ΔC = C_candidat - C_courant
- **Acceptation d'une amélioration :** si ΔC<=0 le chemin candidat est accepté, car son coût est inférieur ou égal à celui du chemin courant**.**
- **Calcul de la probabilité :** si ΔC>0 , calculer la probabilité d'acceptation :

$$P = e^{-\Delta C/T}.$$

- **Prise de décision :** comparer la probabilité (P) à la valeur fixée (r=0,5). Si (r<P), le chemin candidat est accepté ; sinon, il est rejeté et le chemin courant est conservé.
- **Mise à jour de la meilleure solution :** après chaque décision, conserver la meilleure solution rencontrée, c'est-à-dire celle qui possède le coût total le plus faible.

3.5 Tableau des décisions d'acceptation des chemins candidats

Le tableau suivant présente les décisions prises à chaque itération de l'algorithme de recuit simulé.

Itération	ΔC	Probabilité P	Décision
1	-1	—	Accepté, car $\Delta C < 0$
2	$+1$	$e^{-1/8} \approx 0,8825$	Accepté, car $0,5 < 0,8825$
3	$+3$	$e^{-3/6} \approx 0,6065$	Accepté, car $0,5 < 0,6065$
4	-4	—	Accepté, car $\Delta C < 0$
5	$+19$	$e^{-19/2} \approx 0,000075$	Rejeté, car $0,5 > 0,000075$

Les chemins qui améliorent le coût sont acceptés directement. Certains chemins moins performants peuvent aussi être acceptés selon leur probabilité. À la cinquième itération, le chemin est rejeté et la meilleure solution trouvée conserve un coût de 98.

Code python de la methode RS:

TD-H-MH/exercice3/RS.py at master · nourhbaieb88/TD-H-MH